Unit - V

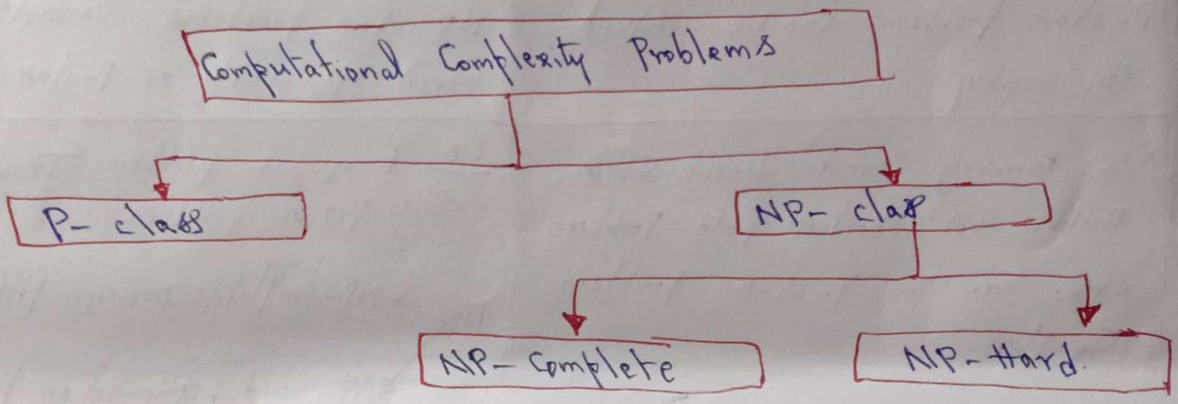
(I) The P-class and NP-class problems

There are two groups in which a problem can be classified. The first group is called P-class problems, in which the problems can be solved in polynomial time. [P → polynomial]

The second group is called NP-class [Non-Deterministic] problems, in which the problems can be solved in non-deterministic polynomial time.

Ex: P-class: Searching the element from the list, Sorting the elements

NP-class: Knapsack problem $O(2^{n/2})$, Travelling Salesperson problem $O(n^2 2^n)$



→ Any problem for which answer is either "yes" or "no" is called "decision problem"

→ Any problem that involves the identification of optimal cost is called Optimal problems

Tractable problems:

The problems that can be solved in polynomial time are called tractable problems

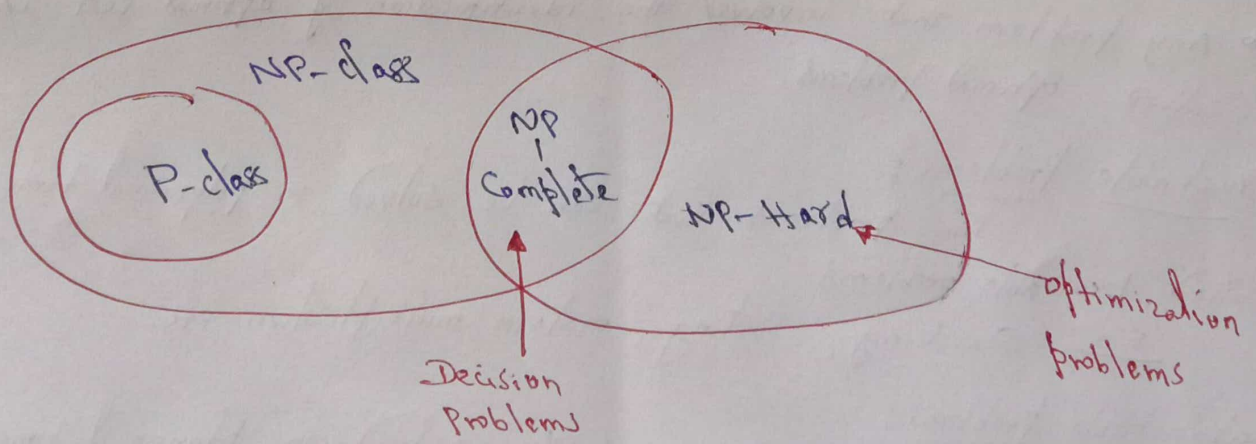
Ex: Searching, Sorting, matrix multiplication etc.

Intractable problems:

The problems that cannot be solved in polynomial time are called intractable problems.

Ex: Tower of Honoi, 8-Queens problem etc.

P-class Problem	NP-class problem
An algorithm in which for a given input, the definite output gets generated is called polynomial time algorithm (P-class)	For a given input there are more than one output gets generated, one cannot determine which output to be selected is called Non-Deterministic polynomial time algorithm (NP-class)
All the P-class problems are basically deterministic	All the NP-class problems are basically Non-Deterministic
Every problem which is in P-class is also in NP-class ($P \subseteq NP$)	Every problem which is in NP is not in the P-class problem
P-class problems can be solved efficiently	NP-class problems cannot be solved efficiently as P-class problems
Ex: Binary Search, Linear Search, Bubble Sort, Shortest Path Problem etc. are the P-class problem examples	Ex: KnapSack Problem, Travelling Sales person problem. NP → NP-Complete: Decision problems; NP-Hard: optimization problems



Non-Deterministic Algorithms

→ The algorithm in which every operation is uniquely defined is called "Deterministic algorithm"

→ The algorithm in which every operation may not have unique results, rather there can be a specified set of possibilities for every operation, such an algorithm is called "non-deterministic" algorithm

Non-Deterministic means that no particular rule is followed to make the guess.

→ The Non-Deterministic algorithm is a two stage algorithm;

(i) Non-Deterministic (Guessing) stage; it generates an arbitrary string that can be thought of as an candidate solution.

(ii) Deterministic (Verification) stage; In this stage, it takes an input the candidate solution and the instance to the problem and returns "yes" if the candidate solution represents actual solution.

Ex:

```
Algorithm Non-Deterministic
    {
    for i := 1 to n do
        A[i] := choose (i)

    if ( A[i] == x) then
        {
            write (1);
            Success() ;
        }
    write (0);
    fail();
    }
```

In the above Non-Deterministic algorithms there are three functions used;

① choose(): Arbitrarily choses one of the element from a given input set.

② Success(): Indicates the Successful Completion

③ fail(): Indicates the un-Successful Completion

Non-Deterministic algorithm for 0/1 knapSack Problem:

```
Algorithm Non-Dknap()
  {
  wt := 0;
  pt := 0;

for i := 1 to n do
   {
   x[i] := choose(0,1);
   wt := wt + x[i] * w[i]; // total weight included
   pt := pt + x[i] * p[i]; // total profit obtained
    }

if ((wt > m) or (pt < profit)) then
                    └→ Target profit
   fail();
else
   Success();

}
```

NP-Hard and NP- Complete problems

→ **NP Problem:** The NP- problems are set of problems whose solutions are hard to find but easy to verify and are solved by Non-Deterministic machine in polynomial time.

→ **NP- Hard problem:**

A problem X is NP-Hard if there is an NP- Complete problem Y, such that Y is reducible to X in polynomial time. NP-Hard problems are as hard as NP- Complete problems. NP-Hard problems need not be in NP- class.

→ **NP- Complete Problem:**

A problem is in NP- Complete if there is an NP- problem Y, such that Y is reducible to X in polynomial time. NP- Complete problems are as hard as NP- problems. A problem is NP- Complete it is a part of both NP and NP- Hard problems. A Non-Deterministic Turing machine can solve NP- Complete problem in polynomial time

NP- Hard	NP- Complete
NP-Hard Problem (say X) can be solved if and only if there is a NP- Complete Problem (say Y) that can be reducible into X in polynomial time	NP complete problems can be solved by a non-deterministic Algorithm/Turing machine in Polynomial Time.
To solve this problem, it do not have to be in NP.	To solve this problem, it must be both NP and NP-hard problems
Do not have to be a decision problem	It is exclusively a Decision problem
Ex: Halting problem, Vertex Cover problem, and all optimization problems	Ex: Determine whether a graph has a Hamiltonian cycle, Determine whether a Boolean formula is satisfiable or not, Circuit- Satisfiability problem etc.

Cook's Theorem

In Computational Complexity theory, the Cook-Levin theorem, also known as Cook's theorem, states that "the Boolean Satisfiability problem is NP-Complete". that is, it is in NP, and any problem in NP can be reduced in polynomial time by a deterministic Turing machine to the Boolean Satisfiability problem.

Theorem: The Satisfiability problem (or Language) (SAT) is NP-Complete

(Proof Outline)

Proof Consists of two steps:

1. Convert the execution of a polynomial time Non-Deterministic Turing machine to a bunch of well formed formulae such that formula Satisfies it and only if the machine accept input
2. Show the sum of lengths of formulae is polynomial in the size of problem

- NP-Hard (L): Can polynomially reduces a any NP problem to L.
- NP complete: $L \in NP$
- $L \in NP$: Non-Deterministic Turing Machine (NDTm) for L that runs in polynomial time.
- $L \in NP \longrightarrow$ NDTM for L that runs in polynomial time
- An NDTm is the only model we have for NP problem
- $SAT \in NP$
- ∴ if we can polynomially reduce and an arbitrary polynomial NDTm to SAT. its mean we have proven SAT is NP-Complete

If we elobarate the above steps we get the proof of Cooks theorem

What is Satisfiability (SAT) problem?

A propositional logic formula ϕ is called satisfiable if there is some assignment to its variables that makes it evaluate to true.

→ $P \wedge q$ is satisfiable; if $p=1$ and $q=1$

→ $P \wedge \neg P$ is not satisfiable

Boolean: (0 or 1)

operators:

NOT ¬

AND ·

OR +

3 SAT:

A Language is $3SAT = \{\phi \mid \phi$ is satisfiable 3-CNF formula$\}$

CNF ⇒ Conjuctive Normal Form

Ex: $(x \vee y \vee z)(\neg x \vee \neg y \vee z)$ It is an 3-CNF.

Every clause has exactly 3 literals (variables $x, y, z, \neg x, \neg y$)

"If there is at least one assignment which satisfies a formula, then we say that this formula is satisfiable, otherwise it is unsatisfiable."

Q:- Prove that the CLIQUE problem is NP Complete.

Ans: For the proof we have to show the below mentioned points:

1. clique
2. 3CNF $\leq_p$ clique
3. clique $\leq_p$ 3CNF $\leq$ SAT
4. clique $\in$ NP

① **clique:** In clique, every vertex is directly connected to another vertex, and the number of vertices in the clique represents the size of clique.

clique Cover: Given a graph G, and an integer k, can we find k subsets of vertices $V_1, V_2, \ldots, V_k$ such that $\cup_i V_i = V$, and that each V_i is a clique of G.

② **3CNF $\leq_p$ clique:**

Proof: For the successful conversion from 3CNF to clique we have to follow the two steps:

Draw the clause in the form of vertices, and each vertex represents a literal of the clauses.

(i) They do not complement each other

(ii) They do not belong to the same clause in the

Conversion, the size of the clique and size of 3CNF must be the same, and we successfully converted 3CNF into clique within the polynomial time.

③ **clique $\leq_p$ 3CNF:**

Proof: As we know that a function of k clause, there must exist a clique of size k. It means that p variables which are from the different clauses can assign the same value (say it is 1). By using these values of all the variables of the CLIQUES, we can make the value of each clause in the function is equal to 1.

Ex: we have a Boolean function in 3CNF:

$(x+y+z)\ (x+y+z')\ (x+y'+z)$

After reduction or conversion from 3CNF to CLIQUE, we will get P variables such as: - $x+y=1$, $x+z=1$, and $x=1$

Put the value of P variables in the equation

$(1+1+0)\ (1+0+1)\ (1+0+1)$

$(1)\ (1)\ (1) = 1$ output Verified

(4) Clique $\in$ NP:-

Proof: As we know very well, we can get the clique through 3CNF and to convert the decision-based NP problem into 3CNF we have to first convert into SAT and SAT comes from NP.

So Concluded that CLIQUE belongs to NP.

Proof of NPC:

(a) Reduction achieved within the Polynomial time from 3CNF to clique.

(b) And verified the output after Reduction from clique to 3CNF above

So, concluded that, if both Reduction and Verification can be done within the Polynomial time that means is clique also in NPC!!